

Game Arcade — build brief

Hand this document to Claude Code (or any engineer) to turn the Game Arcade prototype into an installable Android APK. It specifies the stack, the project layout, the data model, the exact game tuning, the screens to build and the build commands. Read it alongside *Game Arcade — UI build spec*, which owns all visual and copy decisions; this document owns implementation.

Scope of v1: a single-player-feeling app with a simulated social layer. No server, no accounts, no real matchmaking — rivals, friends and the feed are seeded local data. That is deliberate: it makes the retention loop testable on real phones in days, and every seam that later needs a backend is isolated in one module (§7).

1. Stack

Build it with Expo (React Native) and ship the APK with EAS Build. Reasons, in order: EAS produces a real installable APK from one command with no local Android SDK; React Native gives native scroll, gesture and haptics for the shell; and the three games are geometric enough that react-native-reanimated on the UI thread holds 60fps without a game engine. No Unity, no Godot — the games are rectangles.

CONCERN	CHOICE
Framework	Expo SDK (latest stable), TypeScript, React Native
Navigation	expo-router — file-based, with a five-tab layout
Game rendering	react-native-reanimated worklets driving absolutely-positioned <View>s. Add @shopify/react-native-skia only if a later game needs curves or particles.
Gestures	react-native-gesture-handler — tap for STACK/REFLEX, swipe plus buttons for DODGE
State	zustand with an AsyncStorage persist middleware. One store, slices per domain.
Fonts	expo-font loading Archivo 400/600/800 from @expo-google-fonts/archivo
Feel	expo-haptics on every block placement, hit, miss and coin award
Build	EAS Build, preview profile, buildType: apk

The fast alternative, if an APK is needed this week purely for demos: wrap the existing HTML prototype with Capacitor (`npx cap add android`, copy the standalone build into `www/`, `./gradlew assembleDebug`). It produces a working APK in an afternoon, but WebView touch latency is visible in DODGE and the games will feel a half-beat behind. Use it for pitching, not for user testing the loop.

2. Scaffold

```
npx create-expo-app game-arcade --template blank-typescript
cd game-arcade
npx expo install expo-router react-native-safe-area-context react-native-screens \
  expo-linking expo-constants expo-status-bar expo-haptics expo-font \
  @expo-google-fonts/archivo react-native-reanimated react-native-gesture-handler \
  @react-native-async-storage/async-storage
npm i zustand
npx expo install --fix
```

Then: set `"main": "expo-router/entry"` in `package.json`; add `react-native-reanimated/plugin` as the last entry in `babel.config.js`; import `react-native-gesture-handler` on the first line of the root layout.

```
// app.json (essentials)
{ "expo": {
  "name": "Game Arcade", "slug": "game-arcade", "scheme": "gamearcade",
  "orientation": "portrait", "userInterfaceStyle": "light",
  "backgroundColor": "#f3f2f2",
  "android": { "package": "com.gamearcade.app", "versionCode": 1,
    "adaptiveIcon": { "backgroundColor": "#ec3013" } },
  "plugins": ["expo-router", "expo-font"]
}}
```

3. Project structure

```
app/  
  _layout.tsx          root: fonts, gesture handler, store hydration  
  (tabs)/_layout.tsx   five-tab bar (custom, flush-left labels)  
  (tabs)/index.tsx     02 Home – ranked shelf  
  (tabs)/feed.tsx      07 Feed  
  (tabs)/duels.tsx     08 Duels  
  (tabs)/shop.tsx      09 Wallet & shop  
  (tabs)/me.tsx        10 Your profile  
  onboarding.tsx       01 Pick your three  
  browse.tsx           03 Browse all forty  
  game/[id].tsx        04 Game detail  
  match/[id].tsx       05 In-match (no chrome)  
  results.tsx          06 Results + auto-queue  
  drop.tsx / bracket.tsx / friend/[handle].tsx / inbox.tsx  
components/  
  chrome/TopBar.tsx BackBar.tsx TabBar.tsx  
  game/GameRow.tsx ShelfTile.tsx IndexRow.tsx GhostBand.tsx StatStrip.tsx  
  social/FeedPost.tsx DuelRow.tsx LeaderRow.tsx NotificationRow.tsx  
  ui/PosterBand.tsx QueueRing.tsx Toast.tsx Button.tsx Kicker.tsx Figure.tsx  
games/  
  stack/StackGame.tsx reflex/ReflexGame.tsx dodge/DodgeGame.tsx  
  registry.ts          id → component | 'simulated'  
  useRunLoop.ts        shared frame clock, ghost ticker, run lifecycle  
store/  
  index.ts             zustand store + persist  
  slices/games.ts economy.ts social.ts session.ts  
data/  
  catalogue.ts         all 40 game definitions  
  seed.ts              friends, duels, feed, bracket, inbox  
theme/  
  tokens.ts type.ts
```

4. Design tokens

Transcribe these exactly into `theme/tokens.ts` and never write a raw hex in a component.

Radius is zero everywhere — there is no rounded corner in this product.

```

export const C = {
  bg: '#f3f2f2', surface: '#eae9e9', text: '#201e1d',
  accent: '#ec3013', accentPressed: '#dd2b0f',
  accentTint: '#fff2ef', // row background: "a rival is involved"
  accentDeep: '#ae1800', // accent type at 11px and below
  divider: 'rgba(32,30,29,0.4)',
  n200: '#eae7e7', n300: '#d7d3d3', n400: '#bab6b6',
  n500: '#9b9797', n600: '#7d7979', n700: '#605d5d',
  n800: '#444141', n900: '#2d2b2b',
};
export const S = { inset: 14, row: 12, gap: 2, rule: 2, hairline: 1 };
export const T = {
  run: { fontFamily: 'Archivo_800ExtraBold', fontSize: 72, letterSpacing: -2.9 },
  display: { fontFamily: 'Archivo_800ExtraBold', fontSize: 42, letterSpacing: -1.5 },
  title: { fontFamily: 'Archivo_800ExtraBold', fontSize: 26, letterSpacing: -0.8 },
  figure: { fontFamily: 'Archivo_800ExtraBold', fontSize: 25 },
  rowTitle: { fontFamily: 'Archivo_800ExtraBold', fontSize: 17 },
  body: { fontFamily: 'Archivo_400Regular', fontSize: 13, lineHeight: 20 },
  meta: { fontFamily: 'Archivo_400Regular', fontSize: 11 },
  kicker: { fontFamily: 'Archivo_600SemiBold', fontSize: 10, letterSpacing: 1.2 },
};

```

Two rules the prototype breaks and the app must not: every interactive row and tab is at least **48px** tall, and no readable text is below **10px**. Grow padding, never shrink type. All score text uses fontVariant: ['tabular-nums'].

5. Data model

```

type Family = 'TAP' | 'SWIPE' | 'TIMING' | 'NUMBERS' | 'MEMORY';

interface Game {
  id: string; name: string; code: string; // 3 letters, e.g. 'STK'
  family: Family; unit: string; // 'blocks' | 'ms' | 'm' | ...
  lowerIsBetter: boolean; // true for ms
  playable: boolean; // 3 of 40 in v1
  blurb: string; // one sentence, the rule
}

interface Progress { best: number; runs: number; lastPlayed: number | null; pinned: boolean; }
interface Rival { handle: string; score: number; recordedAt: number; isGlobal: boolean; }
interface Run {
  gameId: string; score: number; prevBest: number; improved: boolean;
  beatGhost: boolean; delta: number; nearMiss: boolean; coins: number;
  carriedFrom?: number; variant: boolean; startedAt: number; endedAt: number;
  source: 'shelf' | 'ranked' | 'queue' | 'duel' | 'inbox' | 'feed' | 'poster' | 'retry';
}

interface Wallet { coins: number; freeRetriesLeft: number; retriesResetAt: number; }
interface Session { runs: Run[]; distinctGames: string[]; startedAt: number; }

```

Ranking (implement once, used by the shelf and the queue):

```

rankScore(g) = rivalAhead(g) ? 0 : never Played(g) ? 1 : 2      // ascending
tiebreak     = friendsOn(g) descending
sc-camel-next-in-queue = first ranked game that is not the one just played
reasonLine(g) =
  rivalAhead  → `${rival.handle} BEAT YOU BY ${abs(rival.score - best)}`
  neverPlayed → `${friendsOn} FRIENDS PLAY THIS · YOU NEVER HAVE`
  else       → `QUEST 1/3 · ${friendsOn} FRIENDS ON TODAY`

```

Reason lines render on the top three rows only. Persist: progress per game, wallet, streak, session history, pin set, onboarding completion. Nothing else needs to survive a restart.

6. The three games

One shared hook, `useRunLoop`, owns the frame clock, the ghost ticker and run termination; each game owns only its own mechanic. Run the loop in a `Reanimated` worklet with `useFrameCallback`, clamp `dt` to 48ms, and keep score in a shared value mirrored to `React` state only when it changes.

```

// STACK – TAP, unit: blocks
block.width0 = 140; speed0 = 190 px/s; speed += 9 per placement, cap 430
onTap: overlap = intersect(current, top)
      overlap <= 6 → end run
      |current.x - top.x| < 6 → PERFECT: keep width, score += 2
      else → width = overlap, score += 1
render: last 11 blocks, alternating #201e1d / #444141; active block accent

// REFLEX – TAP, unit: ms (lower is better)
needle sweeps 0..100 at (52 + round * 3.4) units/s, bouncing
band.center = random(18, 82); band.width = max(7, 22 - round * 0.8)
onTap: off = |needle - center|
      off <= width/2 → hit, reported ms = round(78 + off * 16), score += 1
      else → miss; 3 misses end the run
20 rounds max; show round, misses, last reaction under the track

// DODGE – SWIPE + buttons, unit: m
distance += (120 + distance * 0.05) * dt
spawn every max(0.34, 0.72 - distance/4000) s into a random lane of 3
obstacles fall at (250 + distance/9) px/s, 30px tall
player sits 56px above the bottom; same-lane overlap ends the run
score = floor(distance * 10)

// SHARED
ghost.displayed = min(target, round(target * min(1, elapsed / 16)))
coins = max(8, round(score / (unit === 'm' ? 40 : 1.6))) * (variant ? 2 : 1)
sc-camel-near-miss = prevBest > 0 && !improved && delta <= max(2, prevBest * 0.18)
retry: free if freeRetriesLeft > 0 else 50 coins; new run starts at carried score

```

The other 37 games are `playable: false`. Their detail screens are fully real; tapping play runs a four-second simulated run that lands near the rival's score, then produces a genuine Run record so results, coins, feed and queue all behave. Keep this behind one flag so real games can replace it one at a time.

Haptics: light impact on a placement or hit, medium on a perfect, error on a miss, success on a personal best. This is most of what makes a hypercasual game feel expensive.

7. The simulated social layer

Everything social lives in `data/seed.ts` and `store/slices/social.ts`, behind an interface shaped like the eventual network layer — so swapping in a server later is a module replacement, not a rewrite.

- **23 seeded friends** with handles, per-game bests and head-to-head records. Deterministic, not random, so screenshots are reproducible.
- **Ghosts are scores, not replays** in v1 — a number and a timestamp. The header ticker is what sells the race.
- **The feed is derived, not authored.** Every finished run generates a post; seeded friend activity is interleaved by timestamp. When the player beats a rival, generate that friend's "called you out" reply on a delay of a few minutes so the loop closes while they are still in the app.
- **Cold start** creates one named global rival per playable game, matched within $\pm 10\%$ of the player's first score, labelled GLOBAL RIVAL. Never fabricate a friend.
- **Daily reset** at local midnight: free retries back to 3, streak incremented on any run, variant rotated, quests reissued.

8. Build the APK

```
npm i -g eas-cli && eas login
eas build:configure
```

```
// eas.json
{ "cli": { "version": ">= 5.0.0" },
  "build": {
    "preview": { "android": { "buildType": "apk" }, "distribution": "internal" },
    "production": { "android": { "buildType": "app-bundle" } }
  }
}
```

```
eas build -p android --profile preview      # → downloadable .apk URL
# local alternative, needs Android SDK + JDK 17:
npx expo prebuild -p android && cd android && ./gradlew assembleRelease
```

Install with `adb install -r app.apk`, or hand testers the EAS link directly. For side-loading, tell them to enable install-from-unknown-sources; for anything beyond a handful of testers use Play Internal Testing with the production profile (AAB).

9. Build order and acceptance

Ship in this order; each milestone should be installable on a phone before starting the next.

#	MILESTONE	DONE WHEN
1	Shell: tokens, fonts, tab bar, top bar, navigation	All fourteen routes reachable, chrome correct, nothing but placeholders inside.
2	STACK, end to end	Tap-to-drop at 60fps, slicing and perfects correct, run ends, results screen shows a real score, best persists across a cold start.
3	Results, auto-queue and retry	Three-second ring auto-loads the next ranked game; tapping cancels and keeps the suggestion; near-miss band appears only inside 18%; three free retries then a 50-coin charge.
4	REFLEX and DODGE	Both playable with the tuning in §6; ghost ticker running in all three.
5	Catalogue and shelf	Forty games, ranked shelf with reason lines on the top three, Browse with family filters, long-press pinning capped at twelve with the refusal toast.
6	Social layer	Feed derived from runs plus seeded activity, Duels with real ghost scores, friend profiles with head-to-head, inbox actions launching runs directly.
7	Meta and polish	Wallet and shop, streaks, quests, season ladder, daily variant rotation, haptics everywhere, offline queueing.

Acceptance tests for the thesis — the app is not done until these pass on a mid-range Android phone:

1. Cold start to first tap in a game: under **8 seconds**, including the onboarding pick.
2. Run ends to next run begins, unattended: **3 seconds**, with a cancel that always works.
3. Five consecutive queued runs without a dropped frame in DODGE.
4. Kill the app mid-session and relaunch: coins, bests, streak, pins and retry count all intact.
5. Airplane mode: every playable game still plays; results, coins and feed posts appear and reconcile on reconnect.

6. A fresh install with no friends still shows a named rival, a populated global feed and a reason line on every top-three row.

Instrument the events listed in §10 of the UI spec from milestone 3 onward — the queue-acceptance rate and distinct-games-per-session are the whole reason this app exists.

10. Do not

- Round a corner, add a gradient, add a shadow to a row, or introduce a second accent colour.
- Add a splash carousel, a tutorial modal, or a "rate us" prompt.
- Block the player: no interstitial before a run, no forced signup, no mandatory permission on first open.
- Sell access to a game. Coins buy continues and retries only.
- Fabricate friends or fake head-to-head records — seeded rivals must be labelled as global rivals.
- Let the auto-queue run without a visible, tappable cancel. If in doubt, make leaving easier, not harder.

Companion documents: *Game Arcade — UI build spec* (screens, components, copy) and the interactive reference build with STACK, REFLEX and DODGE playable.